

Optimizer Comparison Report

anna.shundeeva

May 2026

1 Introduction

The main goal of this project is to compare two optimizers, AdamW and Muon, as well as a combined approach in which AdamW is used for one subset of the model parameters and Muon is used for another subset.

AdamW is a classical stochastic optimization method widely used for training neural networks. It is known to provide stable results across a wide range of model architectures.

Muon is an alternative optimization approach based on exploiting the matrix structure of gradients. Unlike standard methods, Muon applies a gradient transformation aimed at normalization with respect to the spectral norm, which may potentially improve convergence properties.

The experiments were conducted on the Qwen2.5-0.5B language model using full fine-tuning of the pretrained model. The openwebtext-100k dataset was used as the training dataset.

Model quality was evaluated using a set of benchmark tasks from the lm-evaluation-harness framework: PIQA, ARC (easy and challenge), Winogrande, and HellaSwag.

2 Problem Statement

The goal of this study is to compare the AdamW and Muon optimizers according to the following criteria:

- convergence speed and computational cost during training;
- loss values on the training and validation datasets;
- model quality on downstream tasks.

Let the model be parameterized by a vector of parameters θ . A standard optimization step can then be written as:

$$\theta_{t+1} = \theta_t - \eta \cdot g_t,$$

where η is the learning rate and g_t is the gradient of the loss function.

AdamW uses adaptive estimates of gradient moments, whereas Muon applies a gradient transformation based on its matrix representation. Let G_t be the gradient matrix for parameters that have an explicit matrix structure. Muon then uses a transformation of the following form:

$$G_t = U\Sigma V^T \quad \Rightarrow \quad \tilde{G}_t = UV^T,$$

where $U\Sigma V^T$ is the singular value decomposition of the gradient matrix.

In addition, a combined approach is considered, where the model parameters are split by layers into two subsets:

$$\theta = \theta_{\text{matrix}} \cup \theta_{\text{other}},$$

where the Muon optimizer is applied to θ_{matrix} , while AdamW is applied to θ_{other} . In this experiment, the model was split exactly in half: the first 12 layers were fine-tuned using Muon, while the last 12 layers were fine-tuned using AdamW.

The objective of the experiment is to compare these approaches under identical training conditions.

3 Data

The openwebtext-100k dataset was used as the training dataset. This dataset is already basically preprocessed: it does not contain missing values, extremely short texts, or completely meaningless texts.

Since the main goal of this work is to compare optimizers rather than to study the influence of data quality, additional preprocessing was kept minimal. In particular:

- non-English texts were not removed;
- texts containing HTML markup were kept;
- texts with repeated fragments were not filtered out;
- no selection of texts specifically suited to the validation benchmarks was performed.

This approach reduces the influence of preprocessing on the experimental results and allows the comparison to focus on the optimizers.

All texts were tokenized using the model tokenizer and concatenated into a single token sequence. This sequence was then split into fixed-length blocks with $L = 1024$. A special end-of-text separator token was inserted between individual texts. The remaining tail of the sequence with length less than L was discarded.

The train/validation split was performed randomly with a 95/5 ratio. The resulting split was fixed and used across all experiments.

4 Model

The experiments were conducted using the Qwen2.5-0.5B language model.

This model occupies an intermediate position between the small models used in the original work on the Muon optimizer, such as NanoGPT, and larger language models, such as LLaMA3.2-3B and Qwen2.5-3B. This makes it convenient for comparative experiments under limited computational resources.

The model architecture is based on a standard Transformer block. Let the input to a layer be denoted by x . Then one block can be schematically represented as:

$$x \rightarrow \text{RMSNorm} \rightarrow \text{Attention} \rightarrow \text{Residual} \rightarrow \text{RMSNorm} \rightarrow \text{MLP} \rightarrow \text{Residual}.$$

The self-attention mechanism uses linear projections:

$$q = W_q x, \quad k = W_k x, \quad v = W_v x,$$

where W_q , W_k , and W_v are trainable parameter matrices. The output of the attention transformation is then passed through an additional linear projection:

$$y = W_o \cdot \text{Attention}(q, k, v).$$

The MLP block in the Qwen architecture is implemented using three matrices:

$$h = \text{activation}(W_{\text{gate}} x) \odot (W_{\text{up}} x), \quad y = W_{\text{down}} h,$$

where $\odot$ denotes element-wise multiplication.

Thus, each model layer contains seven parameter matrices with an explicit matrix structure:

$$\{W_q, W_k, W_v, W_o, W_{\text{up}}, W_{\text{gate}}, W_{\text{down}}\}.$$

These parameters are considered candidates for optimization using Muon.

In the Qwen architecture, RoPE (Rotary Positional Encoding) is applied to the q and k vectors. Since RoPE is an orthogonal transformation, its influence on gradient properties and on the effectiveness of Muon optimization is not obvious and requires separate analysis, which was not performed in this work.

5 Experimental Setup

The experiments were conducted under limited computational resources. Model training was performed on an NVIDIA A100 GPU with 40 GB of VRAM, while evaluation on lm-evaluation-harness tasks was performed on an RTX 3070 GPU with 16 GB of VRAM.

To reduce computational cost, the following simplifications were used:

- a small batch size (32);
- a limited number of training steps (2000);

- a single fixed seed for all experiments;
- no compilation with `torch.compile` for the Newton–Schulz iterations.

For each optimizer, the learning rate was selected from a small set of values, while the remaining hyperparameters were kept fixed.

Common optimizer parameters:

- `weight_decay` = 0.01;
- $\beta_1 = 0.9$, $\beta_2 = 0.95$;
- $\varepsilon = 10^{-8}$.

Muon optimizer parameters:

- `momentum` = 0.95;
- `nesterov` = true;
- number of Newton–Schulz iterations: $k = 5$.

The following learning rate values were used:

$$\eta_{\text{AdamW}} \in \{10^{-5}, 3 \cdot 10^{-5}, 5 \cdot 10^{-5}\},$$

$$\eta_{\text{Muon}} \in \{5 \cdot 10^{-5}, 10^{-4}, 3 \cdot 10^{-4}\}.$$

It should be noted that in the Muon implementation, the learning rate is applied only to parameters without matrix structure, which are optimized using AdamW. For matrix-shaped parameters, the update step is determined by the normalized gradient and depends on the matrix dimensions, following the description in the original work.

For the combined approach (layer-wise AdamW + Muon), the learning rate was selected based on preliminary experiments with the individual optimizers:

$$\eta_{\text{combined}} \in \{5 \cdot 10^{-5}\}.$$

6 Results

6.1 Model Quality

The varied parameters, namely the optimizer type and learning rate, together with the main quality metrics, are presented in Table 1.

The final quality metric, Average harness score, was computed as the mean score across the lm-evaluation-harness tasks. For each task, the `acc_norm_none` metric was used when available; otherwise, `acc_none` was used. The full table with detailed metrics is provided in Table 3 in the Appendix.

The pretrained Qwen2.5-0.5 model without fine-tuning was used as the baseline.

Optimizer	LR	Train loss	Val loss	Avg harness score
Base Qwen	-	-	-	0.5365
AdamW	10^{-5}	2.8834	2.8679	0.5470
AdamW	$3 \cdot 10^{-5}$	2.8758	2.8588	0.5460
AdamW	$5 \cdot 10^{-5}$	2.8840	2.8622	0.5407
Muon	$5 \cdot 10^{-5}$	2.8838	2.8640	0.5424
Muon	10^{-4}	2.9380	2.9015	0.5274
Muon	$3 \cdot 10^{-4}$	3.2533	3.1222	0.4570
Combined	$5 \cdot 10^{-5}$	2.8831	2.8626	0.5425

Table 1: Main quality metrics

The AdamW optimizer demonstrates a stable improvement in quality compared to the baseline model, although some fluctuations can be observed in the extended set of metrics. The learning rate $5 \cdot 10^{-5}$ turned out to be too high: both the averaged metric and some individual metrics, especially HellaSwag, noticeably decrease at this value.

The decision to use higher learning rates for Muon turned out to be unsuccessful. The value $3 \cdot 10^{-4}$ substantially worsens both train/validation loss and the final quality metrics. In contrast, $5 \cdot 10^{-5}$ gives stable results on the ARC tasks and even slightly outperforms AdamW according to the averaged harness metric. However, this difference is small and should not be interpreted as a stable advantage without additional runs with different random seeds.

Overall, both optimizers demonstrate similar behavior across different task types; the results do not indicate that either optimizer substantially shifts the model toward better performance on a specific aspect of reasoning.

The layer-wise combined approach demonstrates the expected behavior: its results lie between the corresponding configurations of the pure optimizers. Within the limited scope of this study, this approach did not show a significant improvement in quality compared to AdamW.

6.2 Computational Resources

In the original task statement, the expected advantage of Muon was not necessarily a reduction in the wall-clock time of a single optimization step, but rather an improvement in computational efficiency: achieving comparable or higher model quality with a smaller computational budget. Therefore, in addition to quality metrics, the analysis includes training time and maximum GPU memory usage.

As shown in Table 2, in the conducted experiments Muon uses less GPU memory than AdamW, but is slightly slower. However, it is difficult to draw

Optimizer	LR	Training time (sec)	Max memory (mb)	Avg harness score
AdamW	10^{-5}	5164.29	8803.88	0.5470
AdamW	$3 \cdot 10^{-5}$	5195.90	8803.88	0.5460
AdamW	$5 \cdot 10^{-5}$	5156.04	8803.88	0.5407
Muon	$5 \cdot 10^{-5}$	5402.74	8122.34	0.5424
Muon	10^{-4}	5331.60	8122.34	0.5274
Muon	$3 \cdot 10^{-4}$	5346.36	8122.34	0.4570
Combined	$5 \cdot 10^{-5}$	5276.73	8464.02	0.5425

Table 2: Training time, maximum memory usage, and final quality.

stronger conclusions: the experiments were performed in a short full fine-tuning regime, with a limited number of training steps and without additional computational optimization.

The combined approach occupies an intermediate position: it uses less memory than AdamW but more than Muon, and its training time also lies between the two pure optimizer variants. Within the minimal grid used in this study, it did not show a quality advantage, but it may be interesting as a compromise between quality, training time, and memory usage.

6.3 Training Convergence

Convergence was evaluated based on the dynamics of train loss and validation loss during training. For all successful runs, the loss decreased steadily without signs of divergence. AdamW demonstrated more stable convergence and reached a lower validation loss. Muon was more sensitive to the learning rate: at $3 \cdot 10^{-4}$, a substantial degradation in both loss dynamics and final quality was observed.

7 Conclusion

In this work, a full fine-tuning pipeline for the Qwen2.5-0.5B model was implemented using the AdamW and Muon optimizers, as well as a combined approach in which different layers use different optimization strategies.

The conducted experiments do not allow definitive conclusions to be drawn about the advantage of Muon over AdamW. The main limitations are the small number of runs, the use of a single fixed seed, the limited number of training steps, and an initial learning rate grid for Muon that was not sufficiently well tuned.

Nevertheless, the experiment revealed several interesting observations. First, Muon turned out to be highly sensitive to the choice of learning rate: an overly large value led to a noticeable degradation in both validation loss and downstream metrics. Second, at a comparable learning rate, Muon demonstrated results close to AdamW. Third, the combined approach showed intermediate

behavior between the pure optimizer variants and may be considered a potential compromise between quality, training time, and memory usage.

It was also shown that validation loss on the held-out part of openwebtext-100k does not always directly correspond to model quality on downstream tasks. Therefore, when evaluating optimizers, it is important to consider not only the loss function during training, but also external benchmark metrics.

The following directions can be considered for further development of the experiment:

- extend the learning rate grid for Muon, with more attention to smaller values that are better suited for short fine-tuning of a pretrained model;
- increase the number of training steps and random seeds to evaluate the robustness of the results;
- use computational optimization, for example by restoring `torch.compile` for the Newton–Schulz step, and re-evaluate the wall-clock efficiency of Muon;
- investigate different parameter-splitting strategies for the combined optimizer;
- conduct longer experiments that would make it possible to evaluate the potential computational advantage of Muon on a model at the scale of Qwen2.5-0.5B.

A Appendix

Optimizer	LR	Train loss	Val loss	ARC-C	ARC-C norm	ARC-E	ARC-E norm	HellaSwag	HellaSwag norm	PIQA	PIQA norm	WinoGrande
Base Qwen	-	-	-	0.2892	0.3200	0.6423	0.5816	0.4059	0.5210	0.7046	0.6970	0.5627
AdamW	10^{-5}	2.8834	2.8679	0.2969	0.3234	0.6578	0.6250	0.4048	0.5191	0.7024	0.7008	0.5667
AdamW	$3 \cdot 10^{-5}$	2.8758	2.8588	0.3012	0.3208	0.6671	0.6431	0.4004	0.5101	0.7018	0.6959	0.5604
AdamW	$5 \cdot 10^{-5}$	2.8840	2.8622	0.2884	0.3225	0.6557	0.6330	0.3984	0.5025	0.6959	0.6921	0.5533
Muon	$5 \cdot 10^{-5}$	2.8838	2.8640	0.2944	0.3268	0.6587	0.6296	0.3989	0.5032	0.6970	0.6942	0.5580
Muon	10^{-4}	2.9380	2.9015	0.2747	0.3072	0.6313	0.6090	0.3886	0.4867	0.6926	0.6904	0.5438
Muon	$3 \cdot 10^{-4}$	3.2533	3.1222	0.2116	0.2662	0.5185	0.4764	0.3360	0.3913	0.6447	0.6366	0.5146
Combined	$5 \cdot 10^{-5}$	2.8831	2.8626	0.2884	0.3251	0.6599	0.6296	0.3978	0.5015	0.6986	0.6921	0.5643

Table 3: Full comparison of AdamW, Muon, and the combined approach on the training/validation datasets and lm-evaluation-harness tasks.